

C++ 微積分計算機

從數學邏輯到資料結構的極致效能實踐

資管一 李昱辰

專案介紹

專案目標：建立一套能夠解析算式、執行微分與積分運算、最後化簡結果的微積分引擎

以下是我所預期的功能：

- **算式解析**：將使用者輸入的中序表達式轉換為後序表達式，並進一步建構出系統運算的核心基石——抽象語法樹 (AST)。
- **微分與積分運算**：透過遞迴走訪 AST，結合微積分法則與代數特徵匹配（如變數變換、分部積分），動態生成微積分結果的全新語法樹。
- **自動化簡與有理化**：直接在樹狀結構上進行同類項合併、分數有理化與代數化簡，最後再將結果還原為人類易讀的中序數學式。
- **支援複雜函數**：不僅支援基礎多項式，更涵蓋對數 ($\ln/\log$)、指數 (e^x)、三角函數及反三角函數等超越函數的運算。

系統架構

資料結構底層 (Data Structure)

檔案： `AST.hpp`

負責任務：系統基礎建設，包含全域 Token 結構與二元樹節點 (Node) 定義。

前端解析模組 (Frontend)

檔案： `parser.cpp` , `parser.hpp`

負責任務：詞法分析 (Lexer)、Shunting-Yard 演算法轉換後序式、字串格式化。

核心運算引擎 (Core Engine)

檔案： `calculus.cpp` , `calculus.hpp`

負責任務：AST 語法樹建構、微積分法則遞迴推導、高階演算法派發 (U-Sub, 分部積分)。

系統控制台 (Controller)

檔案： `main.cpp` , `benchmark.cpp`

負責任務：程式進入點、互動計算機模式串接、效能極限壓力測試系統。

資料結構底層 (Data Structure Base)

- **AST 節點結構**：包含 Token 與左右子樹指標。
- **Token 設計**：區分 Operator, Number, Constant, Variable, Function。
- **函數擴充**：支援 sin, ln, arcsin 等超越函數。

```
enum class TokenType
{
    Number,      // 數字 (例如 12, 3.14)
    Variable,    // 變數 (例如 x)
    Operator,    // 運算子 (例如 +, -, *, /)
    LeftParen,   // 左括號 (
    RightParen,  // 右括號 )
    Function,    // 函數 (例如 sin, ln)
    Constant     // 數學常數 (例如 e, pi)
};
```

```
// =====
// 儲存 Token 的結構
// =====
struct Token
{
    TokenType type;
    string value;
    MathFunc funcType = MathFunc::None;
};
```

```
// =====
// 定義 AST (抽象語法樹) 的結構
// =====
struct ASTNode
{
    Token token;
    ASTNode *left;
    ASTNode *right;

    ASTNode(Token t)
    {
        token = t;
        left = nullptr;
        right = nullptr;
    }
};
```

2. 前端解析模組 (Frontend Parser)

將人類易讀的數學字串，轉化為電腦高效運算的抽象語法樹 (AST)

階段一：詞法分析 (Lexer)

- **Token 化處理**：將字串分割為運算元、運算子與函數（如 `sin`，`ln`）。
- **語法正規化**：自動補全隱含乘法（如 `2x` → `2*x`）與處理一元負號（如 `-5`）。
- **錯誤檢測**：識別非法字元並回傳錯誤訊息，防止系統崩潰。

階段二：語法解析 (Parser)

- **Stack 演算法**：實作 Dijkstra 的 Shunting-Yard 演算法，有效處理運算子優先權與括號匹配。
- **中序轉後序 (Infix to Postfix)**：將字串轉換為適合電腦運算的後序式 (Postfix)。
- **AST 建構**：遍歷後序式，透過堆疊操作 (Stack) 動態建構出唯一的二元語法樹。

核心運算引擎 (Core Engine)

微分模組 (Differentiation)

- **Hash Table 查表法**：內建基礎函數微分表，達成 `O(1)` 快速匹配。
- **建立AST**：判斷規則成功後，動態生成全新的 AST 節點並回傳。
- **Chain Rule 自動化**：遞迴處理複雜函數的巢狀微分。
- **線性法則**：完整實作微分四則運算的規則。

積分引擎 (Integration)

- **策略派發機制**：嘗試以下積分技巧：查表、線性規則、變數代換、分部積分、三角代換、部分分式。
- **運作邏輯**：
 - **查表策略**：匹配成功後直接生成並回傳全新的 AST 節點。
 - **遞迴計算**：對複雜技巧執行遞迴推導，直至算出結果或觸發遞迴深度上限。
- **短路回傳**：採用「優先成功制」，任一策略匹配成功即回傳完整 AST；若失敗則回傳 `nullptr`。

系統控制台 (Controller)

互動式 REPL 介面

- **REPL 模式**：提供即時讀取-評估-列印循環，使用者可連續輸入算式進行運算。
- **錯誤優雅處理**：當輸入非法算式時，系統不會崩潰，而是輸出語意化的錯誤提示。

自動化效能 Benchmark

- **壓力測試引擎**：內建多組極限積分測資，自動運算並統計執行時間。
- **效能分析**：輸出精確的微秒級耗時，驗證演算法在極端遞迴下的穩定性。

演算法效能實證：遞迴降次 vs. 代換展開

計算高階三角積分 ($\sec^n \cdot \tan^n$) 時，我使用了代換展開(把算式丟到變數變換法)以及遞迴降次(利用公式解展開算式)兩種方式做運算效能分析，以下是數據結果：

算式複雜度 (n)	代換展開法 (ms)	遞迴降次法 (ms)	效能提升
n=3	3.65	0.59	6.2x
n=11	105.01	14.21	7.4x
n=21	28378.57	3181.49	8.9x

結論： 隨著複雜度 n 增加，遞迴降次法不僅縮短時間，更透過**遞迴步數優化** (由 2357 步降至 107 步)，突破了傳統展開法的運算瓶頸，展現極佳的系統伸縮性。

資料結構的應用

- **Stack (堆疊)**

實作 Shunting-Yard 演算法，精準處理括號與運算優先權，將中序字串完美轉換為後序表達式 (Postfix)。

- **Binary Tree (二元樹)**

專案心臟「抽象語法樹 (AST)」，將複雜的數學分配律與連鎖律，優雅地轉化為樹狀走訪 (Tree Traversal) 與遞迴操作。

- **Red-Black Tree (紅黑樹)**

運用 C++ STL `std::map` 自動排序特性，處理有理函數的降冪排列與長除法，完美解決稀疏多項式的記憶體浪費問題。

- **Hash Table (雜湊表)**

運用 C++ STL `std::unordered_map`，尋找函數的微積分公式時可以以 $O(1)$ 時間找到。